Protiviti MRM Finding Library — Full Documentation

A self-contained web application for managing, clustering, searching, and AI-augmenting **Model Risk Management (MRM)** audit/validation findings. Built as a single Flask backend serving a single-page HTML frontend, with a hand-rolled TF-IDF + bisecting k-means clustering pipeline and a local-LLM (Ollama) integration for regulatory analysis.

Table of Contents

1. Use Case & Problem Statement
2. System Architecture
3. Data Model
4. The Clustering Pipeline
5. Search
6. LLM Integration (Ollama)
7. REST API Reference
8. Frontend Walkthrough
9. Performance & Caching
10. Workflow / End-to-End User Journey
11. File Inventory
12. Setup & Running
13. Configuration
14. Limitations & Design Trade-offs
15. Glossary

---

1. Use Case & Problem Statement

Domain context

Model Risk Management (MRM) is the discipline by which financial institutions identify, measure, monitor, and control risk arising from quantitative models (credit scorecards, IFRS 9 ECL, market risk VaR, capital models, AML, etc.). It is governed by supervisory frameworks such as:

- **SR 11-7** — US Federal Reserve / OCC, Supervisory Guidance on Model Risk Management
- **OCC 2011-12** — Sound practices for model risk management
- **EBA GL 2023/05** — European Banking Authority guidelines

- **BCBS d192** — Basel Committee
- **PRA SS1/23** — UK Prudential Regulation Authority

Audit and validation teams produce **findings** — formal write-ups of issues with a model. A typical finding has a Title, Description, Business Justification, a canonical Theme (e.g. *Model Documentation*, *Model Governance*), and a Suggested Remediation. Across hundreds of engagements, the same problems recur in different language: "no formal data-quality framework", "absence of robust performance monitoring", "weak governance structures", etc.

What the app solves

Pain	How the app addresses it
Reuse: "I'm writing a finding — what similar past findings exist?"	Natural-language **search** over the full library returns the best-matching cluster + ranked findings.
Pattern detection: "What systemic problems recur across our portfolio?"	Auto-clustering groups semantically similar findings and labels each cluster with a dominant theme + top discriminative phrases.
Drafting assistance: "What regulations / validation tests / remediations should accompany this finding?"	An **AI Analyse** action invokes a local Ollama LLM with a strict, MRM-grounded system prompt, producing three deduplicated sections: Regulatory Guidelines, Validation Checks, Remediation Actions.

Who uses it

- Internal audit / Big-4 advisory practitioners drafting MRM reports.
- Validation teams cross-referencing prior findings.
- Demo / PoC for selling MRM tooling capabilities.

---

2. System Architecture

```
┌──────────────────────────────────────────────────────────────────────┐
│  Browser  (index.html — single-page SPA, vanilla JS+Chart.js)        │
│                                                                      │
│  Panels: Dashboard | Findings | Clusters | Search                    │
│          Import | Add | Ollama Logs                                  │
└──────────────────────────────────────────────────────────────────────┘
                           │
                           │ fetch (JSON, gzip-compressed)
                           ▼
┌──────────────────────────────────────────────────────────────────────┐
│  Flask  (index.py — single process, in-memory)                       │
│                                                                      │
│   ┌──────────────┐    ┌──────────────────┐    ┌──────────────────┐  │
```

```
│   │ findings_db   │    │ TF-IDF cache      │    │ search LRU 128 │    │
│   │ (list[dict])  │    │ (post-cluster)    │    │ (query → res)  │    │
│   └───────────────┘    └───────────────────┘    └────────────────┘    │
│                                                                        │
│   ┌───────────────┐                                                    │
│   │ clusters_db   │    Bisecting K-Means + Gap/Silhouette              │
│   │ (list[dict])  │    background daemon thread + lock                 │
│   └───────────────┘                                                    │
│                                                                        │
│  Routes:                                                               │
│   /api/findings  /api/findings/bulk  /api/findings/upload              │
│   /api/findings/<fid>  /api/clusters  /api/clusters/status             │
│   /api/clusters/rerun  /api/search  /api/stats  /api/stats/count│
│   /api/llm  /api/ollama-check  /api/ollama-logs                        │
│                                                                        │
│  Middleware: gzip (≥1 KB JSON/HTML), JSON error handler                │
└────────────────────────────────────────────────────────────────────────┘
                            │
                            │ HTTP (urllib)
                            ▼
                   ┌──────────────────┐
                   │  Local Ollama    │
                   │  127.0.0.1:11434 │
                   │  default model:  │
                   │  tinyllama:latest│
                   └──────────────────┘
```

Key architectural choices

- **No database.** State lives in two Python lists (`findings_db` , `clusters_db`) protected by a `threading.Lock` . Process restart wipes user data; only the 13 hard-coded sample findings reload.
- **No external ML library at runtime.** `scikit-learn` is in `requirements.txt` but never imported. TF-IDF, k-means++, bisecting k-means, silhouette, and gap statistic are all implemented from scratch using pure-Python lists and the `math` module.
- **Single-page app, single file.** The entire frontend (CSS, HTML, JS) lives in `index.html` . No bundler, no framework — vanilla JS plus Chart.js (CDN).
- **Background clustering.** Mutations (`POST` / `DELETE` /upload) return immediately; clustering runs in a daemon thread and the UI polls a lightweight status endpoint to know when it's done.
- **LLM is optional and local.** The `/api/llm` route proxies to Ollama; if unreachable, the rest of the app works fine.

---

3. Data Model

`Finding` (a row in `findings_db`)

Field	Type	Required	Notes
`finding_id`	string	yes	Unique. Duplicates rejected on add/upload.
`title`	string	yes	Short summary. Low clustering weight (generic phrasing).
`description`	string	yes	Long-form description. **Highest** clustering weight.
`business_justification`	string	yes	Risk framing & regulatory context. Second-highest weight.
`suggested_remediation`	string	optional	Action items. Low clustering weight.
`model_theme`	string	optional	Free-text theme; canonicalized via `THEME_GROUPS`. **Not** a clustering signal.

Canonical themes (`THEME_GROUPS`)

Raw input (case-insensitive)	Canonical key
`modelling input data`, `modeling input data`, `model input data`, `input data`	`data_quality`
`model documentation`, `documentation`	`documentation`
`model governance`, `governance`	`governance`
`model methodology`, `methodology`	`methodology`
`model performance`, `performance`	`performance_monitoring`
`model implementation`, `implementation`	`implementation`
`model validation`, `validation`	`model_validation`

`Cluster` (a row in `clusters_db`)

Field	Description
`cluster_id`	`C1`, `C2`, … assigned in dominant-theme order.

Field	Description
`findings_included`	List of `finding_id` strings.
`semantic_label`	Human-readable label, e.g. *"Documentation: development document, model development"*.
`semantic_signals`	Comma-separated list of top discriminative bigrams/unigrams.
`why_grouped`	Auto-generated paragraph explaining the rationale (themes + algorithmic provenance).
`size`	Member count.

---

4. The Clustering Pipeline

End-to-end, the pipeline turns raw text findings into thematic groups using only standard library code. Every stage is hand-implemented in `index.py`.

4.1 Tokenization (`tokenize`, `tokenize_with_bigrams`)

```
re.findall(r'\b[a-z][a-z\-]{2,}\b', text.lower())
```

- Lowercases input.
- Extracts alphabetic tokens of length ≥ 3 (hyphens allowed mid-token).
- Filters against a stop-word list combining English function words **and** MRM domain noise — words like `model`, `risk`, `finding`, `control`, `framework`, `governance`, `policy`, `procedure` that occur in nearly every finding and would otherwise dominate without adding signal.
- Then concatenates **unigrams + bigrams** (`performance_monitoring`, `data_quality`, `version_control`). Bigrams are far more semantically discriminative than the underlying single words.

4.2 Field-weighted token counting (`weighted_token_counts`)

A finding has five text fields contributing different signal density. Each token's count is multiplied by the field weight before vectorization:

Field	Weight	Why
`description`	8	Richest, longest, full of domain vocabulary.

Field	Weight	Why
`business_justification`	7	Risk framing, regulatory consequence language.
`model_theme`	2	Light grouping signal. (Used here, but **not** as a clustering anchor.)
`title`	1	Concise but generic.
`suggested_remediation`	1	Action verbs that recur generically across themes.

> **Important design choice:** `model_theme` does *not* anchor the clustering. Clusters are discovered purely from content, and themes are computed *post-hoc* from the resulting cluster members. This avoids forcing groups into the canonical 7-theme taxonomy when finer or different distinctions exist.

4.3 TF-IDF (`build_tfidf`)

- **Sublinear TF**: `tf = 1 + log(weighted_count)` (dampens dominance of long fields).
- **Smoothed IDF**: `idf = log((N+1)/(df+1)) + 1` (avoids division by zero, mild scaling).
- **Vocabulary pruning**: drops terms that appear in **> 95 %** of documents (universal noise even after stop-word removal).
- **L2 normalization** so cosine similarity reduces to a dot product.

Output: `(matrix, vocab, word2idx)` where `matrix` is a list-of-lists of floats — sparse representation is **not** used; corpora are small (tens to a few thousand findings) so dense lists are simpler and fast enough.

4.4 Auto-selecting `k` (`_auto_select_k`)

When the user doesn't pass an explicit `k`, the algorithm chooses one automatically by combining two well-known criteria.

k-range scaling

```
n ≤ 2  → k = 1
n ≤ 3  → k = 2
n ≤ 10 → k_max = min(n-1, 6)
n ≤ 30 → k_max = min(n/2, 10)
n ≤ 100→ k_max = min(n/4, 15)
n > 100→ k_max = min(n/8, 25)
```

Silhouette

For each candidate k, compute silhouette using bisecting k-means assignments. Silhouette measures how tightly each point fits its cluster vs. the next-best cluster (range −1…1).

Gap statistic (Tibshirani 2001)

For each k, generate `n_refs=5` reference datasets uniformly distributed in the bounding box of the data. The **gap** is `mean(log(W_ref(k))) - log(W_obs(k))`. The Tibshirani rule selects the **smallest k** where `gap(k) ≥ gap(k+1) - sdk(k+1)`.

Combined score

```
score(k) = 0.55 · gap_normalized
         + 0.45 · silhouette
         − 0.002 · k          (complexity penalty)
```

A soft nudge accepts the gap-statistic-preferred k if its combined score is within `0.05` of the maximum — preventing erratic jumps when scores are close.

4.5 Clustering algorithm (`bisecting_kmeans`)

Repeatedly bisects the highest-inertia cluster:

1. Start with one cluster containing all points.
2. Pick the cluster with highest within-cluster inertia.
3. Run flat k-means with `k=2` on its members.
4. Replace that cluster with the two children.
5. Stop when the number of clusters reaches `k`.

The flat `kmeans` helper uses:

- **k-means++ initialization** (sampling proportional to squared cosine distance).
- `n_restarts=3` by default; keeps the best by inertia.
- **Empty-cluster recovery**: re-initializes an empty cluster's centroid to the farthest point from its current assignment.

Bisecting k-means is preferred over flat k-means at higher `k` because it avoids the bad local minima that plague random-init flat k-means with cosine geometry.

4.6 Cluster ordering & labeling

After clustering, clusters are sorted by **dominant canonical theme** in this fixed order so the UI is stable:

```
data_quality → documentation → governance → methodology
→ performance_monitoring → implementation → model_validation
```

`_cluster_label(fids, all_clusters)` produces a human-readable label:

1. Compute the **dominant theme** among members (mode of canonical themes).
2. Compute **discriminative bigrams** via mini TF-IDF: term frequent inside this cluster but rare across other clusters.
3. Output: `"<DominantTheme>: <top bigram>, <second bigram>"` — e.g. `"Performance Monitoring: performance monitoring, monitoring framework"`.

`_top_signals(fids)` returns up to 12 top tokens for the cluster (bigrams preferred), shown in the UI as a chip cloud.

`_generate_why(fids, label)` produces the rationale paragraph displayed for each cluster.

4.7 Concurrency model

```
_cluster_lock = threading.Lock()
_clustering_busy = False  # set during background runs
```

- All write paths (`add_finding`, `bulk_findings`, `upload_findings`, `delete_finding`) call `run_clustering(background=True)` which spawns a daemon thread.
- `_clustering_busy` is polled by the UI via `/api/clusters/status` or `/api/stats/count`.
- The lock guards the swap of `clusters_db[:] = new_clusters` so readers always see a consistent snapshot.
- Manual rerun (`POST /api/clusters/rerun`) runs **synchronously** so the response reflects the new state.
- Initial load at process start runs **synchronously** too — the first request is guaranteed to see clusters.

---

5. Search

`search_query(query)` converts a free-text query into a ranked cluster + finding list.

5.1 Algorithm

1. Tokenize query with the same unigram+bigram pipeline.
2. **Theme boost**: if the query mentions any phrase mapped in `THEME_GROUPS`, inject the theme tokens with weight `0.5` to nudge toward the canonical theme.
3. Reuse the **pre-warmed TF-IDF cache** from the last cluster run (vocab + doc vectors). If the cache is stale (findings list changed mid-flight), rebuild from scratch.
4. Project the query into the existing vocab to get `q_vec` (sublinear-TF, smoothed-IDF, L2-normalized).
5. Cosine similarity to all docs → `sims` list.

6. **Best cluster = highest *average* member similarity**, not the cluster of the single nearest doc. This finds the most *thematically* relevant group rather than chasing a single outlier.
7. Return `{cluster, findings, score}` where `findings` are the cluster's members ranked by individual similarity.

5.2 Search cache

```
_search_cache          # dict, max 128 entries
_SEARCH_CACHE_MAX = 128
```

- Keyed on the raw query string.
- Evicts the oldest half on overflow (simple and fast — no need for a true LRU).
- Invalidated whenever findings or clusters change.

---

6. LLM Integration (Ollama)

6.1 Why Ollama?

- Runs locally — no API keys, no data leaves the machine, free for demo use.
- Default model `tinyllama:latest` (~1.1 B params) gives serviceable MRM analysis at low resource cost.
- Easily swappable via env: `OLLAMA_MODEL=llama3.1:8b ollama serve`.

6.2 The proxy route (`/api/llm`)

```
POST /api/llm
{ "system": "...", "user": "..." }
→ { "text": "..." }   |   { "error": "..." } (HTTP 502)
```

- Tries each candidate host in order: `OLLAMA_HOST` env → `127.0.0.1:11434` → `localhost:11434`.
- Calls Ollama's `/api/chat` with `stream:false`, `num_predict:512`, `temperature:0.3`.
- 180 s timeout (model warm-up can be slow).
- Logs every step into a 200-line ring buffer, surfaced via `/api/ollama-logs`.
- Returns helpful, actionable error messages when Ollama is unreachable (`ollama serve` / `ollama run <model>` instructions) or timing out.

6.3 Frontend AI Analyse flow (`doLLMAnalysis`)

After a search, clicking **AI Analyse** issues three sequential LLM calls, each with a strict system prompt and deduplication:

1. **Regulatory Guidelines** (5–6 numbered points) — citations across SR 11-7, EBA GL 2023/05, BCBS d192, OCC 2011-12, PRA SS1/23. Each point: regulation + section + obligation paragraph.
2. **Validation Checks** (5–7 points) — concrete tests with evidence + pass/fail criteria.
3. **Remediation Actions** (5–7 points) — corrective actions with implementation method + expected risk-reduction outcome.

Each section seeds the next with its token sets; **Jaccard similarity** between numbered points across sections is computed and any pair above `0.40` is collapsed to one — preventing cross-section duplication ("Regulatory" repeating the same content as "Remediation").

The base system prompt enforces strict output rules:

- Numbered list only (1. 2. 3. ...), no resets.
- No quoting or paraphrasing of background context.
- No mention of finding IDs, titles, themes, or field names.
- No headers, intros, markdown, or closing remarks.
- One self-contained sentence/paragraph per item.

6.4 Per-finding AI panes (`generateFindingPane`)

Each finding card has on-demand **Validation** and **Regulatory** generation buttons that produce a 3–6 item analysis specific to that one finding, rendered as styled prose blocks (accent dot + thin divider, no numbered badges).

---

7. REST API Reference

All endpoints return JSON. Errors return `{ "error": "..." }` with appropriate 4xx/5xx status. Responses ≥ 1 KB are gzip-encoded when the client sends `Accept-Encoding: gzip`.

Static

Method	Path	Description
`GET`	`/`	Serves `index.html` (no Jinja parsing — JS template literals are passed through verbatim).

Findings

Method	Path	Body	Description

Method	Path	Body	Description
`GET`	`/api/findings`	—	Returns the full findings list.
`POST`	`/api/findings`	`{finding_id, title, description, business_justification, model_theme?, suggested_remediation?}`	Add one finding. 409 on duplicate ID. Triggers background cluster.
`POST`	`/api/findings/bulk`	`[{...}, {...}]`	Bulk JSON insert. Returns `{added, skipped, total}`.
`POST`	`/api/findings/upload`	`multipart/form-data` (`file=*.xlsx|*.csv`)	Parses with pandas/openpyxl. Flexible column resolution: e.g. `Finding ID`, `findingid`, `id` all map to `finding_id`.
`DELETE`	`/api/findings/<fid>`	—	Remove one finding. Triggers re-cluster.

Clusters

Method	Path	Body	Description
`GET`	`/api/clusters`	—	Returns all clusters.
`GET`	`/api/clusters/status`	—	`{busy, count}` — lightweight poll for "still clustering?".
`POST`	`/api/clusters/rerun`	`{k?}`	**Synchronously** re-clusters. If `k` omitted, auto-selects.

Search

Method	Path	Body	Description
`POST`	`/api/search`	`{query: "..."}`	Returns `{cluster, findings, score}`.

Stats

Method	Path	Description
`GET`	`/api/stats`	`{total_findings, total_clusters}`.
`GET`	`/api/stats/count`	`{f, c, busy}` — minimal payload for high-frequency polling.

LLM (Ollama)

Method	Path	Body	Description
`POST`	`/api/llm`	`{system, user}`	Proxies to Ollama. Returns `{text}` on success, `{error}` (502) on failure.
`GET`	`/api/ollama-check`	—	Probes each candidate host, returns `{host: {reachable, models}}` for diagnostics.
`GET`	`/api/ollama-logs`	—	`{lines, status}` — last 200 log lines + reachability ping. Polled every 2 s by the Logs panel.

---

8. Frontend Walkthrough

A single `index.html` (~3,500 lines) with seven panels selected via the left navigation.

8.1 Dashboard (`renderDashboard`)

- KPI tiles: total findings, total clusters, average cluster size.
- **Donut chart** — distribution by canonical theme (Chart.js, palette `CHART_PALETTE`).
- **Bar chart** — cluster sizes.

8.2 Findings Library (`renderFindings` , `renderFindingsPage`)

- Paginated table (`PAGE_SIZE = 25`).
- Local filter (`filterFindings`).
- Per-row expand → full description + business justification + remediation, plus per-finding **AI Validation / AI Regulatory** buttons.
- Delete action → `DELETE /api/findings/<fid>` → triggers polling banner.

8.3 Clusters (`renderClusters`)

- Cards in dominant-theme order with the `semantic_label`, `why_grouped`, `semantic_signals` chips, and member finding tiles.
- Cluster colors cycle through 8 palette entries (`CLUSTER_COLORS`).
- **Re-cluster** button → `POST /api/clusters/rerun`.

8.4 Search & Query (`doSearch`, `doLLMAnalysis`)

- Free-text input → `POST /api/search`.
- Result page: best-matched cluster card + list of related findings (sorted by similarity).
- **AI Analyse** button → three-tab output (Regulatory / Validation / Remediation).

8.5 Bulk Import (`importCSV`, `uploadFile`)

- Drag-and-drop file zone (`fileDropOver`, `fileDropped`) for `.xlsx` / `.csv`.
- Tab to paste raw JSON (`switchTab`).
- Tab to paste CSV text directly.

8.6 Add Finding (`addFinding`)

- Form with all six fields. Required-field validation. Clears on success.

8.7 Ollama Logs

- Connection status badge (polled every 2 s).
- Active host, loaded models, last-checked timestamp tiles.
- Live log window (terminal-styled, dark theme) showing the rolling 200-line buffer with optional auto-scroll and live-polling toggles.

8.8 Cross-cutting UI primitives

- **Cluster banner** (`showClusterBanner` / `hideClusterBanner`) — bottom-center spinner shown while `_clustering_busy` is true.
- **Global search dropdown** (`globalSearch`) — top-bar fuzzy autocomplete across all findings.
- **Modal** — used for finding detail overlays.
- **Alert zone** — non-blocking toast notifications.
- **Jaccard text dedup** (`_tokenSet`, `_jaccard`, `deduplicateSection`) — frontend dedup of LLM output across sections.

---

9. Performance & Caching

9.1 Gzip middleware (`compress_response`)

- Triggers on text-type responses (`json` , `html` , `javascript` , `text`) ≥ 1 KB when client accepts gzip.
- `compresslevel=6` (good size/speed balance).

9.2 TF-IDF cache (`_tfidf_cache`)

- Built immediately after every successful clustering run (`_prewarm_tfidf_cache`).
- Stores `{vectors, vocab, word2idx, token_counts, fid_snapshot}` .
- `fid_snapshot` is the ordered finding-ID list at build time; on every search this is compared to the current order. If unchanged, the search reuses the matrix and projects the query in O(|query_tokens|). If different (rare race condition mid-mutation), falls back to a full rebuild.

9.3 Search cache (`_search_cache`)

- Dict keyed on raw query, capped at 128 entries.
- Eviction: when full, drops the **oldest half** (cheap pseudo-LRU).
- Invalidated on every findings/cluster mutation.

9.4 Cheap polling

- `/api/stats/count` returns `{f, c, busy}` — three integers + a bool. Designed to be polled at sub-second intervals without overhead.
- `/api/clusters/status` similarly minimal.

9.5 Cluster work off the request thread

Mutations return in milliseconds. Heavy cluster computation runs in `threading.Thread(daemon=True)` — the UI animates a banner and polls until `busy=false` .

---

10. Workflow / End-to-End User Journey

```
┌──────────────────────────────────────────────────────────────┐
│ START                                                        │
│ Server boots → loads 13 SAMPLE_FINDINGS                      │
│ → blocking initial cluster pass → first request sees clusters │
└──────────────────────────────────────────────────────────────┘
        │
        ▼

┌──────────────────────┐
│ User adds findings   │
│  - single (form)     │
```

```
│  - bulk JSON paste    │
│  - xlsx / csv upload │
└──────────────────────┘
        │
        │ (each mutation)
        ▼

┌──────────────────────────────────────────────────────────┐
│ Cache invalidation (TF-IDF + search)                     │
│ Background daemon thread: _do_clustering                 │
│   1. weighted_token_counts per finding                   │
│   2. build_tfidf                                         │
│   3. _auto_select_k (gap + silhouette + penalty)         │
│   4. bisecting_kmeans                                    │
│   5. theme-ordered cluster list                          │
│   6. _cluster_label / _top_signals / _generate_why       │
│   7. swap clusters_db; pre-warm TF-IDF cache             │
│ UI polls /api/clusters/status, banner hides on done.     │
└──────────────────────────────────────────────────────────┘
        │
        │
        ▼

┌──────────────────────────────────────────────────────────┐
│ User explores                                            │
│  Dashboard — donut + bar charts                          │
│  Findings  — paginated, filter, delete, expand           │
│  Clusters  — themed, labeled, with rationale             │
│  Search    — free-text → best cluster + ranked findings  │
└──────────────────────────────────────────────────────────┘
        │
        │
        ▼

┌──────────────────────────────────────────────────────────┐
│ AI Analyse (after a search)                              │
│  - 3 sequential calls to /api/llm:                       │
│     (1) Regulatory Guidelines (5-6 pts)                  │
│     (2) Validation Checks     (5-7 pts)                  │
│     (3) Remediation Actions   (5-7 pts)                  │
│  - cross-section Jaccard dedup at threshold 0.40         │
│  - Each call seeded with prior section's token sets      │
└──────────────────────────────────────────────────────────┘
        │
        │
        ▼
   Optional: per-finding AI buttons (Validation / Regulatory)
```

---

11. File Inventory

File	Lines	Role

File	Lines	Role
`index.py`	1,408	Flask app — routes, ML pipeline, Ollama proxy, sample data, error handlers, gzip middleware.
`index.html`	3,506	Entire SPA — CSS (Protiviti palette), HTML markup, vanilla JS application logic.
`Protiviti-logo-1.png`	—	Sidebar logo.
`requirements.txt`	6	`flask`, `gunicorn`, `numpy`, `scikit-learn`, `pandas`, `openpyxl`. (`numpy` / `scikit-learn` are unused at runtime — pandas is used for xlsx/csv parsing.)

Major functions in `index.py`

Function	Line	Role
`compress_response`	25	gzip middleware
`tokenize` / `tokenize_with_bigrams`	172 / 178	text → tokens
`weighted_token_counts`	205	per-finding weighted bag-of-words
`build_tfidf`	220	sublinear-TF / smoothed-IDF / L2-normalized vectors
`kmeans`	270	flat k-means++ with cosine geometry
`bisecting_kmeans`	331	recursive bisection clustering
`_silhouette_score`	381	sampled silhouette
`_gap_statistic`	440	Tibshirani gap
`_auto_select_k`	517	combined gap + silhouette − penalty
`_cluster_label`	631	semantic label generator
`_top_signals`	691	discriminative tokens
`_generate_why`	715	rationale paragraph

Function	Line	Role
`_do_clustering`	735	full cluster pass
`_prewarm_tfidf_cache`	815	post-cluster cache build
`run_clustering`	836	sync/async dispatch
`search_query`	851	query → best cluster + ranked findings
`index`	944	serve `index.html`
`add_finding` / `bulk_findings` / `upload_findings` / `delete_finding`	960–1101	mutation routes
`get_clusters` / `clusters_status` / `rerun_clustering`	1110–1126	cluster routes
`search`	1130	search route
`stats` / `stats_count`	1141–1152	stats routes
`_log` / `_ollama_hosts` / `_ollama_model` / `_try_ollama_chat`	1164–1232	Ollama plumbing
`ollama_check` / `ollama_logs` / `llm_proxy`	1234–1331	Ollama-facing routes

---

12. Setup & Running

Prerequisites

- Python 3.9+
- (Optional) Ollama for the AI Analyse feature.

Install

```
pip install -r requirements.txt
```

Run

```
python index.py
# → http://0.0.0.0:5000
```

The dev server binds to `0.0.0.0:5000` with `debug=True` and `use_reloader=False`. For production-style runs, use `gunicorn`:

```
gunicorn -w 1 -b 0.0.0.0:5000 index:app
```

> **Always run with a single worker (`-w 1`).** State is in-process Python lists; multiple workers would each have their own findings/clusters and updates would not be visible across them.

Enable the AI Analyse feature

In a separate terminal:

```
ollama serve
ollama run tinyllama:latest    # or any model you prefer
```

Then visit `http://localhost:5000/api/ollama-check` to confirm reachability. The Ollama Logs panel in the UI shows live activity.

---

13. Configuration

All configuration is via environment variables — there is no config file.

Variable	Default	Purpose
`OLLAMA_HOST`	(unset → falls back to `127.0.0.1:11434`, then `localhost:11434`)	Override the Ollama base URL.
`OLLAMA_MODEL`	`tinyllama:latest`	The Ollama model used for `/api/llm`.

Internal "knobs" are constants in `index.py`:

Constant	Value	Where

Constant	Value	Where
`FIELD_WEIGHTS`	`{description:8, business_justification:7, model_theme:2, title:1, suggested_remediation:1}`	top of file
`STOP_WORDS`	English + MRM noise	top of file
`THEME_GROUPS`	canonical theme map	top of file
`_SEARCH_CACHE_MAX`	128	cache module
Gap statistic refs	5	`_gap_statistic`
K-means restarts	3	`kmeans`
K-means max iter	100	`kmeans`
Auto-k score weights	`0.55*gap + 0.45*sil - 0.002*k`	`_auto_select_k`
Cross-section dedup threshold (frontend)	`0.40`	`doLLMAnalysis`
LLM `num_predict`	512	`_try_ollama_chat`
LLM temperature	0.3	`_try_ollama_chat`
LLM timeout	180 s	`_try_ollama_chat`
Log buffer size	200 lines	`_log_buffer`
Page size (UI)	25	`index.html`

---

14. Limitations & Design Trade-offs

Limitations

- **Volatile state.** Restart wipes all user-added findings. Suitable for demos; would need a database for production.
- **Single-worker only.** Concurrency is in-process; multi-worker WSGI deployments would split state.
- **No auth.** Anyone with network access to the port can read, add, or delete findings.

- **Pure-Python ML.** Fast enough for thousands of findings but won't scale to tens of thousands without moving to NumPy/scikit-learn.
- **English-only tokenization.** Stop words and bigram heuristics assume English text.
- `tinyllama` **quality ceiling.** The default LLM is small; for richer regulatory analysis swap in a larger model.
- **Sample data is hardcoded.** 13 sample findings are appended on every boot, so user-deleted samples come back at restart.

Why these trade-offs

- The codebase is optimized for **readability, demoability, and zero external dependencies at runtime**. Every piece of the pipeline is auditable in one file. This is uncommon in MRM-adjacent tooling and itself part of the value proposition.
- **In-memory state** makes it trivially deployable (`python index.py` is the install).
- **Bisecting k-means + auto-k** trades cluster count flexibility for stable, reproducible groupings without operator tuning.
- **Local LLM** keeps sensitive audit content off third-party APIs.

---

15. Glossary

- **MRM** — Model Risk Management. The discipline of identifying, measuring, monitoring and controlling risk arising from quantitative models in financial institutions.
- **Finding** — A formal write-up of a deficiency in a model, produced by audit/validation/regulators.
- **Theme** — Canonical category of a finding (e.g. *Documentation*, *Governance*).
- **TF-IDF** — Term Frequency × Inverse Document Frequency. A weighting scheme that boosts terms frequent in a document but rare across the corpus.
- **Cosine similarity** — Angle-based similarity between two vectors; ranges 0…1 for non-negative vectors. Equivalent to dot product on L2-normalized vectors.
- **K-Means / K-Means++** — Centroid-based clustering. K-Means++ improves initialization by sampling seeds proportional to squared distance from existing seeds.
- **Bisecting K-Means** — Hierarchical variant that repeatedly splits the worst-fit cluster in two; more stable for higher k.
- **Silhouette** — Cluster-quality score in [−1, 1] measuring how well each point fits its cluster vs. the next-best.
- **Gap statistic** — Tibshirani's method for choosing k by comparing observed within-cluster dispersion to that expected from random reference data.
- **Bigram** — A two-word sequence (`performance_monitoring`); typically more discriminative than the underlying unigrams.
- **Sublinear TF** — `1 + log(tf)` rather than raw `tf` ; dampens dominance of very long documents.
- **Ollama** — Local LLM server. Hosts open-weight models (`llama3.x` , `tinyllama` , etc.) on `127.0.0.1:11434` .
- **Jaccard similarity** — `|A ∩ B| / |A ∪ B|` over token sets; used here to dedup LLM output.

- **SR 11-7** — US Federal Reserve / OCC Supervisory Letter on Model Risk Management (2011). The foundational MRM framework in the US.

Documentation generated against the codebase at the current snapshot. If you change `FIELD_WEIGHTS`, the auto-k formula, or any route, update the corresponding section above.